

C2F1G4 Speaker Script: ArchiHotel Reservation Handling & Check-In

Final presentation · Mon 06.07.2026, 16:30 · Room C.0.44 · Total ≈ 12:15 min + 5 min Q&A

Each block matches the part of the model the speaker built. Time per person is roughly two minutes.
[CLICK] means press for the next highlight animation. [NEXT SLIDE] means advance to the next slide.
Speak the text, don't read it word for word. It is written the way you would say it.

Speaker	Slides	Part	Time
Katrin Schwab	1-4	Intro + abstract model	~2:15
Soyeong Jeong	5-7	Business layer	~2:20
Fiona Aurelia Tunggalmuljo	8	Application layer I	~1:40
Mingwai Kelly Bin	9	Application layer II	~1:40
Demir Eroglu	10	Technology layer	~2:00
Angel Choi	11-13	Reflection + summary	~2:20

Everyone stays up front for the discussion. They will ask each team member something.

Katrin: Slides 1 to 4 (~2:15)

Slide 1 · Title (0:15)

Good afternoon everyone! We are team C2F1G4, and we spent the last weeks inside ArchiHotel, a luxury hotel group whose promise is that luxury feels effortless. Here is the thing though: behind that effortless feeling, five systems have to cooperate for every single check-in. That's what we modeled.

[NEXT SLIDE]

Slide 2 · Our EAM Team (0:20)

Quick word on how we worked: one layer per person. Soyeong took the business layer, Fiona and Kelly built the application layer together, Demir went down into the technology layer, Angel made sure all the layers actually fit together, and I built the abstract view and did the final quality pass. Six people, one model.

[NEXT SLIDE]

Slide 3 · The Case (0:30)

So what does ArchiHotel actually do? Four things matter for us today. Guests book through the website or through external platforms. At the desk, reception checks your identity and hands you a key card. Payment runs through an external provider. And every morning the staff aligns on one shared picture of the day. Everything you see in the red box, reservation handling and check-in, is our focus area F1. That's the journey from booking to arrival.

[NEXT SLIDE]

Slide 4 · The Abstract EA Model (1:10)

This is the whole company on one slide, and please, don't try to read it. It's the map, not the content.

[CLICK] What you should see is the structure: three layers. The yellow band on top is business, everything people do. The blue band is application, the software. The green band at the bottom is technology, the hardware and networks it all runs on.

[CLICK] Two people drive our focus area: the Guest up here on the left, and the Receptionist, who runs the check-in.

[CLICK] And one thing to remember for the rest of the talk: almost every arrow eventually points at this box, the Hotel Management System, which runs on the AWS Cloud Server. If you remember one box, remember this one.

Soyeong will now zoom us in.

Soyeong: Slides 5 to 7 (~2:20)

Slide 5 · Focus Area F1 (0:30)

Thanks Katrin. Here is the same map again, and the red box is where we zoom in: the chain from Handle Reservation to Conduct Check-In/Out, with everything underneath that supports it. We'll do this in three steps. First the business layer, who does what. Then the application layer, which systems support it. And finally the technology layer, what it all runs on.

[NEXT SLIDE]

Slide 6 · Business Layer: Reservation (0:55)

Let's start where the guest starts.

[CLICK] The Guest triggers everything, either on our website or on an external booking platform.

[CLICK] In the abstract view this was one box called Handle Reservation. In the detailed model it becomes seven concrete steps: submit the request, verify room availability, calculate the price, confirm and allocate the room, process the payment, submit special requests, and finally add everything to the arrival schedule.

[CLICK] Payment is its own business service, and it's delivered by an external payment provider.

[CLICK] And each step reads or writes a business object down here: the reservation itself, room availability, pricing, open costs, the arrival schedule.

[NEXT SLIDE]

Slide 7 · Business Layer: Check-In (0:55)

A few days later the guest walks in. Now it's the Receptionist's turn.

[CLICK] She is assigned to the whole check-in process.

[CLICK] Three steps: identify the reservation, verify the guest's identity, and issue and configure the key card.

[CLICK] Together these three steps realize the Check-In/Out Service, which is what the guest actually experiences at the desk.

[CLICK] And any open costs settle through the same payment service we just saw.

So much for the people. Fiona will show you the systems underneath.

Fiona: Slide 8 (~1:40)

Slide 8 · Application Layer: Inside the HMS (1:40)

Thank you Soyeong. Kelly and I modeled what happens below the surface, and honestly, the abstract view hid a lot.

[CLICK] Remember the one box to remember? Here it is again: the Hotel Management System. On this slide we open it up.

[CLICK] Inside, its functions are modeled explicitly. Check Room Availability, Calculate Pricing, Reserve Room, Issue Key Card. Each of these is a real piece of functionality, not just a label.

[CLICK] And these functions don't stay inside. They are exposed upward as application services: Reservation Management, Check-In, Key Verification. Every service you see here serves one of the business processes Soyeong just walked you through. That's the link between the two layers.

Kelly will now show you the dynamic side: what actually happens when one reservation flows through this system.

Kelly: Slide 9 (~1:40)

Slide 9 · Application Layer: Events & Data (1:40)

Thanks Fiona. Same picture, different question: what happens at runtime?

[CLICK] It starts here: a Reservation Request Received event fires, and the system processes the reservation data.

[CLICK] Then follow the chain to the right. The reservation gets approved, payment is processed, Payment Confirmed fires, and only after payment the special request goes through. So the order of events is modeled, not just the parts.

[CLICK] One more pattern we want to show you, because it repeats everywhere: realization. This Reservation data object is the digital record behind the Reservation business object from slide 6. The business world talks about a reservation, the system stores one, and the dashed arrow connects the two. Guest profiles, pricing, room status, they all follow exactly the same pattern.

And with that, down to the metal. Demir?

Demir: Slide 10 (~2:00)

Slide 10 · Technology Layer: Down to the Door (2:00)

Thanks Kelly. My favorite question of the whole project was: what happens when the guest actually stands in front of the door? So let's open a door together.

[CLICK] The guest taps the key card, and the Door Card Reader detects it.

[CLICK] That kicks off the Grant Room Access chain, five technology functions in a row: detect the card, read the access rights, verify the authorization, unlock the door, and activate the room circuit, so the lights come on.

[CLICK] The Door System Software works with two artifacts: it reads the Access Rights and it writes an Access Log entry. So every door opening leaves a trace.

[CLICK] Communication is modeled explicitly too: RFID between card and reader, and hotel Wi-Fi and Internet up to the HMS on AWS.

[CLICK] And that closes the loop of our story: a reservation made online, weeks earlier, becomes a door that opens. That's the whole architecture working together. Angel will wrap up with what we learned.

Angel: Slides 11 to 13 (~2:20)

Slide 11 · Reflection: Value & Limitations (0:55)

Thanks Demir. So, was building this model worth it? Three reasons why we'd say yes. First, transparency: five cooperating systems, one shared picture, no tribal knowledge. Second, impact analysis. You can literally trace the arrows and ask: if the HMS goes down, what stops working? The model answers that. Third, onboarding: a new receptionist or a new developer sees exactly where their work sits.

But let's be honest about the limits. It's a snapshot, and nothing in it keeps itself current. It has no quantities, no SLAs, so capacity questions stay open. And every abstraction choice hides detail, that's the price of readability.

[NEXT SLIDE]

Slide 12 · Reflection: Missing Information & Challenges (0:55)

Some things we simply couldn't model, because the case material didn't tell us. There are no infrastructure specs for the reservation side, the interfaces to the external booking platforms stay vague, and there's no failure or volume data at all.

And our real challenges? Choosing the abstraction level, when is a box one system or five? Keeping the abstract and the detailed view consistent while both kept changing. And, I can personally confirm this one: modeling in parallel with coArchi gives you merge conflicts and duplicate elements. At one point we merged eleven duplicates in one sitting. The kickoff slides warn about exactly this. They were right.

[NEXT SLIDE]

Slide 13 · Summary (0:30)

So, to sum up. We modeled reservation handling and check-in across all three ArchiMate layers, and everything converges on the Hotel Management System. We showed you the map first, then one story per layer, from a click on the website down to a door that opens. The value: transparency and impact analysis. The limits: it's a snapshot, without quantities.

Thank you very much! We're happy to take your questions.

(Keep slide 13 up during the discussion. Backup slides 15 to 17 have the full views and model facts if a question needs them.)

Q&A prep (5 min, everyone)

- **Full detailed view** → backup slide 15. **Full abstract view** → backup slide 16. **Numbers and workflow** → backup slide 17.
- Likely questions per person: Katrin (why this abstraction level, audit findings), Soyeong (why seven steps, process vs. function), Fiona (why explicit interfaces, HMS decomposition), Kelly (event chain

order, realization pattern), Demir (why model RFID, artifacts vs. data objects), Angel (merge workflow, how layers were aligned).